

LabadaFlow — Master Specification

> **Authoritative reference for all LabadaFlow implementation decisions.**

> Produced from all locked decisions, architectural discussions, and product constraints.

> Another competent AI engineer should be able to implement LabadaFlow from this document alone.

1. Product Vision and Scope

Product: LabadaFlow

Tagline: Every load has a flow.

LabadaFlow is a web-based Laundry Management System built as a polished SaaS-style academic project. It is not a commercial platform. It is a focused, complete, defensible MVP with excellent UX, architecture, security, and presentation quality.

Core experience: Track laundry orders from intake to release through a defined operational workflow.

RECEIVED → WASHING → DRYING → FOLDING → READY → COMPLETED

The system manages:

- Laundry customers (walk-in and registered)
- Services with configurable pricing
- Orders with itemized line items
- Order workflow with an immutable audit trail
- Staff and admin team access
- A self-service customer portal
- Public order tracking via unguessable code

Academic defense goal: The system must be fully demonstrable in a concise end-to-end walkthrough and explainable by students who built it.

2. Locked Technology Stack

These are permanently locked. Do not replace or add alternatives.

| Concern | Technology |

|---|---|

| Framework | Next.js (App Router) |

| Language | TypeScript |

| Styling | Tailwind CSS |

| Component foundation | shadcn/ui |

| Authentication / identity | Clerk |

| Database | Supabase PostgreSQL |

| Micro-interactions | Magic UI |

| Marketing visual effects | Aceternity UI (selective, marketing only) |

| Icons | Lucide React |

Do NOT introduce:

- Drizzle ORM or any other ORM
- A separate REST API or backend service
- React Query or SWR (unless a concrete streaming/real-time requirement appears)
- Global client state management (Zustand, Redux, Jotai)
- Stripe, GCash, Maya, PayPal, or any payment provider
- Prisma

Data access: Supabase JS client (`@supabase/ssr`) called server-side only, via a typed data access layer. No ORM. Parameterized queries via Supabase's query builder or `rpc()` for transactions.

...

3. Architecture

Application structure

Single Next.js App Router application. No separate API server.

```

labadaflow/
  app/
    (marketing)/
      page.tsx → /
    (auth)/
      login/page.tsx → /login
    (public)/
      track/
        page.tsx → /track
        [code]/page.tsx → /track/[code]
    (backoffice)/
      layout.tsx (auth guard, role guard, sidebar)
      dashboard/page.tsx
      orders/
        page.tsx
        new/page.tsx
        [orderId]/
          page.tsx
          slip/page.tsx
      customers/
        page.tsx
        [customerId]/page.tsx
      services/page.tsx
      team/page.tsx
      reports/page.tsx
      settings/page.tsx
    (portal)/
      layout.tsx (auth guard, customer-role guard)
      portal/
        page.tsx (redirects to /portal/orders)
        orders/
          page.tsx
          [orderId]/page.tsx
          profile/page.tsx
  components/
    ui/ (shadcn generated – do not hand-edit)
    app/ (shared product components)
      order-flow-stepper.tsx
      status-badge.tsx
      empty-state.tsx
      page-header.tsx
    marketing/ (marketing-only components)
  lib/
    data/ (data access layer)
      customers.ts
      orders.ts
      services.ts
      team.ts
    actions/ (Server Actions)
      customers.ts
      orders.ts
      services.ts
      team.ts
    validation/ (Zod schemas)
      customer.ts
      order.ts
      service.ts
    supabase/
      server.ts (server client factory)
      database.types.ts (generated types from supabase gen types)
      money.ts (centavo ↔ peso conversion)
      time.ts (Asia/Manila date utilities)
      order-machine.ts (state machine transition logic)
      tracking.ts (nanoid tracking code generation)
  middleware.ts (Clerk auth guard)
  supabase/
    migrations/
      001_initial_schema.sql
    seed.ts

```

Data flow — reads

- URL / page request
- Next.js Server Component
- lib/data/[domain].ts (DAL function)
 - auth() from Clerk → get userId
 - query users table → get actor role
 - authorize operation for that role
 - scoped Supabase query (select only required columns)
 - return role-shaped DTO
- render

Data flow — mutations

```
Client form submit
→ Server Action (lib/actions/[domain].ts)
→ auth() → get userId
→ Zod schema validation
→ get actor from users table → verify role
→ authorize the specific operation
→ enforce domain/business rules (state machine, etc.)
→ Supabase mutation (transaction via rpc() where atomicity needed)
→ revalidatePath / revalidateTag
→ return { success: true } or { error: string }
→ UI feedback
```

Supabase client

```
// lib/supabase/server.ts
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export function createClient() {
  const cookieStore = cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { ... } }
  )
}
```

The anon key is safe for server-side use because Supabase RLS is not the security model here — the DAL is. The service role key is never exposed to the client.

4. Clerk Authentication Architecture

What Clerk handles

- User registration and login (email/password)
- Session management and JWTs
- User metadata (role storage)
- Admin invitation flow for new team members and customers

Role storage

Roles are stored in Clerk `publicMetadata`:

```
{ "role": "ADMIN" }
// or "STAFF" or "CUSTOMER"
```

`publicMetadata` can only be written server-side (via Clerk Backend API or in Server Actions with the service key). It is safe to read in session tokens because it cannot be set by the client.

Users table in Supabase

A `users` table mirrors Clerk users who are backoffice team members or registered portal customers. It is the persistent identity record for audit trails and team management UI.

```
users.clerk_user_id (TEXT, UNIQUE) → links to Clerk user
users.role → source of truth for authorization (must match Clerk metadata)
```

When admin updates a user's role via `/team`, the Server Action:

1. Updates `users.role` in Supabase
2. Calls Clerk Backend API to update `publicMetadata.role`

Both must succeed; if Clerk update fails, roll back the Supabase update.

Middleware

`middleware.ts` uses Clerk's `authMiddleware` to:

- Protect `/dashboard`, `/orders`, `/customers`, `/services`, `/team`, `/reports`, `/settings`
- Protect `/portal/*`
- Allow public access to `/`, `/login`, `/track`, `/track/*`

Middleware is NOT the security boundary. It is a UX redirect layer. Every protected Server Component and Server Action re-authenticates independently.

Authentication flow by user type

Admin/Staff:

1. Admin logs in at `/login` → Clerk session established
2. Middleware checks auth → allows access to backoffice
3. Layout checks role $\geq$ STAFF → renders backoffice UI
4. Individual pages/actions re-check role for their specific operation

Customer (portal):

1. Customer registers via `/login` (self-service, Clerk)
2. Admin sets their role to CUSTOMER via `/team` (or via Clerk dashboard)
3. Customer logs in → middleware routes to `/portal`
4. At first portal load, DAL looks up `customers.clerk_user_id = userId`
5. If not linked: system checks `customers.email` matching user's primary email → auto-links if unique match found → updates `customers.clerk_user_id`
6. If no match: customer sees "Contact the shop to activate your account."
7. If linked: render their portal

Environment variables

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY    (server-only, never exposed to client)
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY
CLERK_SECRET_KEY              (server-only)
CLERK_WEBHOOK_SECRET          (if webhooks used)
```

5. Supabase Architecture

Access pattern

- **Client type:** `@supabase/ssr` server client, created per-request in Server Components and Server Actions
- **No RLS:** Row Level Security is intentionally not used. The DAL is the access control layer.
- **No ORM:** Direct query builder calls and `rpc()` for transactions
- **Connection pooling:** Use Supabase's built-in connection pooler (Transaction mode for Server Actions, Session mode for long operations if needed)
- **Type safety:** `supabase gen types typescript` generates `database.types.ts` from the schema

Transactions

Multi-step atomic operations (e.g., complete order) use Postgres functions called via `supabase.rpc()`:

```
// Example: complete_order Postgres function handles:
// UPDATE orders SET status='COMPLETED', paid_at=NOW(), completed_at=NOW()
// INSERT INTO order_status_events (...)
// Both in one transaction
await supabase.rpc('complete_order', { p_order_id: orderId, p_actor_id: actorId, p_actor_name: actorName })
```

Order number generation

A Postgres sequence generates order numbers:

```
CREATE SEQUENCE order_number_seq START 1;
```

Application generates: `'LF-' || LPAD(nextval('order_number_seq')::TEXT, 5, '0')`

Produces: LF-00001, LF-00002, ... LF-99999

Tracking code generation

Generated application-side using `nanoid` (16 characters, URL-safe alphanumeric). Stored as TEXT with a UNIQUE constraint. Probability of collision is negligible for MVP scale.

6. Role and Permission Model

Roles

Role	Description
ADMIN	Full access including team management, service config, and all data
STAFF	Operational access — customers, orders, workflow
CUSTOMER	Portal access — own orders and profile only

Permission matrix

Operation	ADMIN	STAFF	CUSTOMER
View dashboard	✓	✓	—
View all orders	✓	✓	—
Create order	✓	✓	—
Advance/backstep order status	✓	✓	—
Cancel order	✓	✓	—
Complete order (set paid_at)	✓	✓	—
View order slip	✓	✓	—
Create/edit customer	✓	✓	—
Archive/restore customer	✓	—	—
View own customer record	✓	✓	✓ (portal)
Create/edit services	✓	—	—
Activate/deactivate service	✓	—	—
Manage team	✓	—	—
View reports	✓	✓	—
View settings	✓	—	—
Portal: own orders	—	—	✓
Portal: own profile	—	—	✓
Public tracking	public	public	public

Authorization enforcement

Authorization is enforced inside every DAL function and every Server Action — never in middleware alone. The pattern:

```
async function getActor(userId: string) {
  const supabase = createClient()
  const { data } = await supabase
    .from('users')
    .select('id, role, full_name, is_active')
    .eq('clerk_user_id', userId)
    .single()
  if (!data || !data.is_active) return null
  return data
}

// In every protected Server Action:
const { userId } = await auth()
if (!userId) throw new Error('UNAUTHENTICATED')
const actor = await getActor(userId)
if (!actor) throw new Error('FORBIDDEN')
if (!['ADMIN', 'STAFF'].includes(actor.role)) throw new Error('FORBIDDEN')
```

7. Customer vs Clerk-User Distinction

This is a foundational design constraint. **Clerk user ≠ laundry customer.**

| Concept | Description |

|---|---|

| Clerk user | An authenticated application identity (staff, admin, or registered customer) |

| customers record | A laundry business customer — a person who brings laundry |

Walk-in customers are customers records with no Clerk account. They are fully supported. Staff creates their record when they first bring laundry.

Registered customers have both a customers record and a Clerk account. Their `customers.clerk_user_id` is populated once the link is established.

The link is established by email match at first portal login (system queries `customers.email` against user's Clerk email and auto-links if a unique, unlinked match exists).

Rules:

- Never assume `auth().userId` maps to a customers record
- Never query orders by `clerk_user_id` directly — always go through `customers.id`
- Customer portal queries: `WHERE customer_id = (SELECT id FROM customers WHERE clerk_user_id = $userId)`
- A customers record with no `clerk_user_id` can still have orders, history, and full staff-managed data

8. Complete Database Schema

Enum types

```
CREATE TYPE user_role AS ENUM ('ADMIN', 'STAFF', 'CUSTOMER');
CREATE TYPE order_status AS ENUM ('RECEIVED', 'WASHING', 'DRYING', 'FOLDING', 'READY', 'COMPLETED', 'CANCELLED');
CREATE TYPE order_source AS ENUM ('WALK_IN', 'PORTAL');
CREATE TYPE pricing_unit AS ENUM ('PER_KG', 'PER_PIECE');
```

Table: `users`

Represents backoffice team members and registered portal customers who have Clerk accounts.

```
CREATE TABLE users (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_user_id TEXT        NOT NULL UNIQUE,
  role        user_role     NOT NULL DEFAULT 'STAFF',
  full_name   TEXT          NOT NULL,
  email       TEXT          NOT NULL,
  is_active   BOOLEAN       NOT NULL DEFAULT true,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
```

Table: `customers`

Represents laundry business customers. Walk-ins have no `clerk_user_id`.

```
CREATE TABLE customers (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_user_id TEXT        UNIQUE,          -- nullable, set when portal account is linked
  full_name   TEXT          NOT NULL,
  phone       TEXT,          -- not unique; warn on duplicate, do not block
  email       TEXT,
  address     TEXT,
  notes       TEXT,
  archived_at TIMESTAMPTZ,    -- soft delete
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
```

Table: `services`

Laundry service types with pricing configuration.

```
CREATE TABLE services (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT          NOT NULL,
  description TEXT,
  pricing_unit pricing_unit NOT NULL,
  unit_price_cents INTEGER   NOT NULL CHECK (unit_price_cents >= 0),
  is_active   BOOLEAN       NOT NULL DEFAULT true,
  sort_order  INTEGER       NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
```

Sequence: `order\_number\_seq`

```
CREATE SEQUENCE order_number_seq START 1 INCREMENT 1;
```

Table: `orders`

```
CREATE TABLE orders (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  order_number      TEXT                NOT NULL UNIQUE DEFAULT ('LF-' || LPAD(nextval('order_number_seq')::TEXT, 5, '0')),
  tracking_code     TEXT                NOT NULL UNIQUE,
  customer_id       UUID                NOT NULL REFERENCES customers(id),
  status            order_status       NOT NULL DEFAULT 'RECEIVED',
  source            order_source       NOT NULL DEFAULT 'WALK_IN',
  assigned_to       UUID                REFERENCES users(id),
  created_by        UUID                NOT NULL REFERENCES users(id),
  total_cents       INTEGER             NOT NULL DEFAULT 0 CHECK (total_cents >= 0),
  due_at            TIMESTAMPTZ,
  notes            TEXT,
  cancel_reason     TEXT,
  received_at       TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  paid_at           TIMESTAMPTZ,
  completed_at      TIMESTAMPTZ,
  cancelled_at      TIMESTAMPTZ,
  created_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW(),

  CONSTRAINT completed_requires_payment CHECK (
    status <> 'COMPLETED' OR (paid_at IS NOT NULL AND completed_at IS NOT NULL)
  ),
  CONSTRAINT cancelled_requires_reason CHECK (
    status <> 'CANCELLED' OR (cancel_reason IS NOT NULL AND cancel_reason <> '')
  ),
  CONSTRAINT paid_at_only_on_completion CHECK (
    paid_at IS NULL OR status = 'COMPLETED'
  ),
  CONSTRAINT cancelled_at_matches_status CHECK (
    (status = 'CANCELLED') = (cancelled_at IS NOT NULL)
  )
);
```

Table: `order\_items`

Price fields are snapshotted at item creation. Historical orders are unaffected by service price changes.

```
CREATE TABLE order_items (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id          UUID                NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  service_id        UUID                REFERENCES services(id) ON DELETE SET NULL,
  service_name      TEXT                NOT NULL, -- snapshot
  pricing_unit      pricing_unit       NOT NULL, -- snapshot
  unit_price_cents  INTEGER             NOT NULL CHECK (unit_price_cents >= 0), -- snapshot
  quantity          NUMERIC(10,3)      NOT NULL CHECK (quantity > 0),
  line_total_cents  INTEGER             NOT NULL CHECK (line_total_cents >= 0),
  created_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW()
);
```

quantity is NUMERIC(10,3) to support both PER_KG (e.g., 2.500 kg) and PER_PIECE (e.g., 5.000 pieces). line_total_cents = ROUND(unit_price_cents × quantity), computed server-side.

Table: `order\_status\_events`

Append-only audit trail. Never update or delete rows.

```
CREATE TABLE order_status_events (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id          UUID                NOT NULL REFERENCES orders(id),
  actor_id          UUID                REFERENCES users(id) ON DELETE SET NULL,
  actor_name        TEXT                NOT NULL, -- snapshot of actor's name at event time
  from_status       order_status,       -- NULL for the initial RECEIVED event
  to_status         order_status       NOT NULL,
  note             TEXT,               -- required for backward transitions
  created_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW()
);
```

9. Relationships, Constraints, and Indexes

Relationships summary

users (1) ■■< orders (created_by)	one staff creates many orders
users (1) ■■< orders (assigned_to)	one staff assigned to many orders
customers (1) ■■< orders	one customer has many orders
orders (1) ■■< order_items	one order has many items
orders (1) ■■< order_status_events	one order has many status events
services (1) ■■< order_items	one service referenced by many items (nullable after delete)
users (1) ■■< order_status_events	one actor produced many events

Indexes

```
-- orders
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created_at ON orders(created_at DESC);
CREATE INDEX idx_orders_assigned_to ON orders(assigned_to);
CREATE INDEX idx_orders_received_at ON orders(received_at DESC);
-- order_number and tracking_code are UNIQUE → already indexed

-- order_items
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_order_items_service_id ON order_items(service_id);

-- order_status_events
CREATE INDEX idx_events_order_id ON order_status_events(order_id, created_at ASC);

-- customers
CREATE INDEX idx_customers_phone ON customers(phone);
CREATE INDEX idx_customers_email ON customers(email);
CREATE INDEX idx_customers_archived_at ON customers(archived_at);
CREATE INDEX idx_customers_full_name ON customers USING gin(to_tsvector('simple', full_name));
-- clerk_user_id is UNIQUE → already indexed

-- users
-- clerk_user_id is UNIQUE → already indexed
CREATE INDEX idx_users_role ON users(role);
```

Postgres functions for atomic operations

```
-- complete_order: atomically sets COMPLETED + paid_at + completed_at + inserts status event
CREATE OR REPLACE FUNCTION complete_order(
  p_order_id UUID,
  p_actor_id UUID,
  p_actor_name TEXT
) RETURNS VOID AS $$
BEGIN
  UPDATE orders
  SET status = 'COMPLETED',
      paid_at = NOW(),
      completed_at = NOW(),
      updated_at = NOW()
  WHERE id = p_order_id
  AND status = 'READY';

  IF NOT FOUND THEN
    RAISE EXCEPTION 'Order not in READY status or does not exist';
  END IF;

  INSERT INTO order_status_events (order_id, actor_id, actor_name, from_status, to_status)
  VALUES (p_order_id, p_actor_id, p_actor_name, 'READY', 'COMPLETED');
END;
$$ LANGUAGE plpgsql;
```

10. Complete Order State Machine

Valid transitions

RECEIVED	→	WASHING	(forward)
WASHING	→	DRYING	(forward)
DRYING	→	FOLDING	(forward)
FOLDING	→	READY	(forward)
READY	→	COMPLETED	(forward, completion action – see Section 12)
WASHING	→	RECEIVED	(backward, requires note)
DRYING	→	WASHING	(backward, requires note)
FOLDING	→	DRYING	(backward, requires note)
READY	→	FOLDING	(backward, requires note)
RECEIVED	→	CANCELLED	(cancellation, requires reason)
WASHING	→	CANCELLED	(cancellation, requires reason)
DRYING	→	CANCELLED	(cancellation, requires reason)
FOLDING	→	CANCELLED	(cancellation, requires reason)
READY	→	CANCELLED	(cancellation, requires reason)

Terminal statuses: COMPLETED and CANCELLED. No transitions are allowed from these statuses.

COMPLETED → **anything**: INVALID

CANCELLED → **anything**: INVALID

Transition validation logic

Encoded in `lib/order-machine.ts`:

```
const FORWARD_TRANSITIONS: Record<string, string> = {
  RECEIVED: 'WASHING',
  WASHING: 'DRYING',
```

```

    DRYING:    'FOLDING',
    FOLDING:   'READY',
    READY:    'COMPLETED', // handled by complete_order, not generic advance
  }

  const BACKWARD_TRANSITIONS: Record<string, string> = {
    WASHING: 'RECEIVED',
    DRYING:  'WASHING',
    FOLDING: 'DRYING',
    READY:   'FOLDING',
  }

  export function validateTransition(from: OrderStatus, to: OrderStatus) {
    if (from === 'COMPLETED' || from === 'CANCELLED') {
      return { valid: false, reason: 'Terminal status cannot transition' }
    }
    if (to === 'CANCELLED') {
      return { valid: true, isBackward: false, isCancellation: true }
    }
    if (FORWARD_TRANSITIONS[from] === to) {
      return { valid: true, isBackward: false, isCancellation: false }
    }
    if (BACKWARD_TRANSITIONS[from] === to) {
      return { valid: true, isBackward: true, isCancellation: false }
    }
    return { valid: false, reason: `${from} → ${to} is not a valid transition` }
  }
}

```

This logic is called by Server Actions. The server always validates the transition regardless of what the client sent.

Item editing window

Order items can only be added, edited, or removed while the order is in RECEIVED status. Once the order advances to WASHING, items are locked. This is enforced in the Server Action.

11. Status History and Audit Model

`order\_status\_events` behavior

- **Append-only.** No row is ever updated or deleted.
- **First event:** Created when the order is created. `from_status = NULL`, `to_status = 'RECEIVED'`.
- **Subsequent events:** Every transition (forward, backward, cancellation, completion) appends a row.
- **Actor snapshot:** `actor_name` is snapshotted at event creation so historical records remain accurate if the actor is later deactivated.
- **Note:** Required for backward transitions. Optional otherwise.

Display

- Staff/admin: Full status history shown on the order detail page as a timeline.
- Customer portal: Status history shown on the portal order detail page (same data, customer-facing labels).
- Public tracking: Only current status shown. No actor information, no notes.

12. Real-World Payment Confirmation Behavior

LabadaFlow does not process payments. There are no payment gateways, payment methods, payment transaction tables, refund flows, partial payments, or reconciliation. This is a permanently locked product decision.

What "payment" means in this system:

A staff member or admin physically receives cash (or any real-world payment method) from the customer when they pick up their laundry. The staff member then marks the order as complete in LabadaFlow.

The completion action is atomic and single-step:

```

Staff clicks "Complete & Mark Paid" on a READY order
→ Server Action: complete_order
→ auth() + authorize (ADMIN or STAFF)
→ verify current status = READY (server-side, from DB)
→ call complete_order() Postgres function:
    UPDATE orders SET status='COMPLETED', paid_at=NOW(), completed_at=NOW()
    INSERT INTO order_status_events (from='READY', to='COMPLETED')
→ revalidatePath('/orders/[orderId]')
→ UI reflects COMPLETED state

```

Fields used:

- orders.total_cents — the amount to collect, displayed to staff at completion
- orders.paid_at — timestamp set atomically at completion (NOT set independently of COMPLETED)
- orders.completed_at — timestamp set atomically at completion

DB constraint enforces this:

```
CONSTRAINT completed_requires_payment CHECK (  
    status <> 'COMPLETED' OR (paid_at IS NOT NULL AND completed_at IS NOT NULL)  
)  
CONSTRAINT paid_at_only_on_completion CHECK (  
    paid_at IS NULL OR status = 'COMPLETED'  
)
```

paid_at is never set on a non-COMPLETED order. The system cannot record payment without completing the order.

13. CRUD Requirements

Customers

- | Operation | Who | Notes |
- |---|---|---|
- | Create | Staff, Admin | Duplicate phone warning (not block) |
- | List | Staff, Admin | Search by name/phone, filter archived, paginate |
- | View | Staff, Admin, Customer (own) | |
- | Edit | Staff, Admin | |
- | Archive | Admin only | Soft delete via archived_at |
- | Restore | Admin only | Clear archived_at |
- | Delete | NEVER | Preserve for order history integrity |

Duplicate phone detection: when creating a customer, if another active customer with the same phone exists, show a warning dialog but allow staff to proceed.

Services

- | Operation | Who | Notes |
- |---|---|---|
- | Create | Admin only | |
- | List | Admin, Staff (for order creation) | Active services sorted by sort_order |
- | Edit | Admin only | Price changes do not affect existing orders (snapshots) |
- | Toggle active | Admin only | Inactive services hidden from order creation |
- | Reorder | Admin only | Update sort_order |
- | Delete | NEVER | Deactivate instead (preserves order_items.service_id references) |

Orders

- | Operation | Who | Notes |
- |---|---|---|
- | Create | Staff, Admin | Customer required, at least 1 item required |
- | List | Staff, Admin | Filter by status, date, customer; search by order number; paginate |
- | View | Staff, Admin, Customer (own) | |
- | Add/edit items | Staff, Admin | Only while status = RECEIVED |
- | Advance status | Staff, Admin | Forward one step; validates via state machine |
- | Backstep status | Staff, Admin | Backward one step; requires note |
- | Cancel | Staff, Admin | Requires reason; any non-terminal status |
- | Complete | Staff, Admin | Only from READY; atomic paid_at + completed_at |

| Print slip | Staff, Admin | /orders/[orderId]/slip — printable layout |

| Hard delete | NEVER | Cancel instead |

Team (users)

| Operation | Who | Notes |

---|---|---

| List | Admin only | |

| Invite new member | Admin only | Sends Clerk email invite; admin sets initial role |

| Edit role | Admin only | Updates Supabase + Clerk publicMetadata atomically |

| Deactivate | Admin only | Sets is_active = false; does not delete |

| Reactivate | Admin only | |

| Delete | NEVER | Preserve for order/event history integrity |

Status events

Append-only. No create/edit/delete UI. Written automatically by Server Actions.

14. Route Structure

Public / unauthenticated

| Route | Access | Description |

---|---|---

| / | Public | Marketing homepage |

| /login | Public | Clerk sign-in UI |

| /track | Public | Order tracking search form |

| /track/[code] | Public | Order status by tracking code |

Backoffice (requires ADMIN or STAFF)

| Route | Description |

---|---

| /dashboard | Summary stats, recent orders |

| /orders | Orders list with filters/search/pagination |

| /orders/new | Create new order form |

| /orders/[orderId] | Order detail — status, items, history, actions |

| /orders/[orderId]/slip | Printable order slip |

| /customers | Customers list with search/pagination |

| /customers/[customerId] | Customer detail — info, order history |

| /services | Services list and management (Admin: edit; Staff: read) |

| /team | Team management (Admin only) |

| /reports | Operational reports (Admin and Staff: view) |

| /settings | System settings (Admin only) |

Customer portal (requires CUSTOMER role + linked customer record)

| Route | Description |

---|---

| /portal | Redirects to /portal/orders |

| /portal/orders | Customer's own orders list |

| /portal/orders/[orderId] | Customer's own order detail and status history |

| /portal/profile | Customer profile (view/edit own info) |

URL parameter conventions

- Lists use query params: ?search=, ?status=, ?page=, ?sort=
- Filters are URL-driven (no client state for filters)
- Pagination: ?page=1 (default 20 per page)

Tracking page behavior

- /track/[code] where code does not exist returns same UI as "not found" (no 404 distinction — prevents enumeration)
- Tracking response: order status label, received date, estimated ready date (if due_at set). No customer PII. No actor info.

15. Server Component / Server Action / DAL Pattern

Server Components

All backoffice pages and portal pages are Server Components by default. Interactive islands (forms, dialogs, status action buttons) are Client Components composed within the Server Component tree.

```
// app/(backoffice)/orders/[orderId]/page.tsx
export default async function OrderDetailPage({ params }: { params: { orderId: string } }) {
  const order = await getOrderById(params.orderId) // DAL call
  if (!order) notFound()
  return (
    <div>
      <OrderFlowStepper currentStatus={order.status} />
      <OrderItemsTable items={order.items} />
      <StatusHistory events={order.statusEvents} />
      <OrderActions order={order} /> {/* Client Component */}
    </div>
  )
}
```

Server Actions

Located in lib/actions/. Every action follows the canonical pattern:

```
'use server'

export async function advanceOrderStatus(orderId: string, toStatus: string, note?: string) {
  // 1. Authenticate
  const { userId } = await auth()
  if (!userId) return { error: 'Unauthenticated' }

  // 2. Validate input
  const parsed = AdvanceStatusSchema.safeParse({ orderId, toStatus, note })
  if (!parsed.success) return { error: 'Invalid input' }

  // 3. Get actor + authorize
  const actor = await getActorByClerkId(userId)
  if (!actor || ![ 'ADMIN', 'STAFF' ].includes(actor.role)) return { error: 'Forbidden' }

  // 4. Get current order (DB is source of truth)
  const order = await getOrderForMutation(parsed.data.orderId)
  if (!order) return { error: 'Order not found' }

  // 5. Validate state machine
  const transition = validateTransition(order.status, parsed.data.toStatus as OrderStatus)
  if (!transition.valid) return { error: transition.reason }

  // 6. Business rules
  if (transition.isBackward && !parsed.data.note?.trim()) {
    return { error: 'Note required for backward transition' }
  }

  // 7. Mutate
  const supabase = createClient()
  await supabase.from('orders').update({ status: parsed.data.toStatus, updated_at: new Date().toISOString() })
    .eq('id', parsed.data.orderId)
  await supabase.from('order_status_events').insert({
    order_id: parsed.data.orderId,
    actor_id: actor.id,
    actor_name: actor.full_name,
    from_status: order.status,
    to_status: parsed.data.toStatus,
    note: parsed.data.note ?? null,
  })

  // 8. Revalidate
  revalidatePath(` /orders/${parsed.data.orderId}`)

  return { success: true }
```

```
}
```

DAL functions

Located in `lib/data/`. Shape data for the calling role. Never return more fields than necessary.

```
// lib/data/orders.ts
export async function getOrderById(orderId: string): Promise<OrderDetailDTO | null> {
  const { userId } = await auth()
  if (!userId) return null

  const actor = await getActorByClerkId(userId)
  if (!actor) return null

  const supabase = createClient()
  const { data } = await supabase
    .from('orders')
    .select(`
      id, order_number, tracking_code, status, source, total_cents, due_at, notes,
      received_at, paid_at, completed_at, cancelled_at, cancel_reason,
      customer:customers(id, full_name, phone, email),
      assigned_staff:users!assigned_to(id, full_name),
      items:order_items(id, service_name, pricing_unit, unit_price_cents, quantity, line_total_cents),
      events:order_status_events(id, from_status, to_status, note, actor_name, created_at)
    `)
    .eq('id', orderId)
    .single()

  if (!data) return null

  // Role shaping: customers see reduced data
  if (actor.role === 'CUSTOMER') {
    const customerRecord = await getCustomerByClerkId(userId)
    if (!customerRecord || customerRecord.id !== data.customer.id) return null
    return shapeForCustomer(data)
  }

  return shapeForStaff(data)
}
```

16. Security and IDOR Prevention

Core security requirements

Every protected Server Action and DAL function MUST:

1. Authenticate the actor (Clerk `auth()`)
2. Verify actor exists in users table and `is_active = true`
3. Verify actor's role permits the operation
4. Scope the query to data the actor is allowed to see
5. Return only required fields (role-shaped DTO)
6. Validate all input (Zod)
7. Enforce business rules server-side (state machine, business constraints)

IDOR prevention

Customer portal:

All customer portal data access must filter through the customer's own record:

```
// WRONG:
.from('orders').select('*').eq('id', orderId)

// CORRECT:
const customer = await getCustomerByClerkId(userId)
if (!customer) return null
.from('orders').select('*').eq('id', orderId).eq('customer_id', customer.id)
```

Public tracking:

- Respond identically for non-existent and found-but-unauthorized tracking codes
- Return only: current status label, received date, due date (no customer PII, no actor names, no notes)

Order not found:

Unauthorized access to a customer's order via the portal returns the same UI as a genuinely missing order (prevents existence leakage).

Attack vectors defended

| Vector | Defense |

|---|---|

| IDOR on orders | Portal queries always filter by `customer_id = currentCustomer.id` |

| Privilege escalation | Role read from Supabase users table server-side, never from client |

| Role spoofing | `publicMetadata.role` writable only server-side via Clerk Backend API |

| Malformed state transitions | State machine validated server-side on every transition attempt |

| Duplicate mutation / race condition | DB constraint on `status` + conditional UPDATE (`WHERE status = expected`) |

| Injection | Supabase parameterized query builder; Zod schema validation on all inputs |

| Secret exposure | `SUPABASE_SERVICE_ROLE_KEY` and `CLERK_SECRET_KEY` are server-only env vars |

| Tracking enumeration | `/track/[code]` returns identical 404-like response for missing and unauthorized codes |

| Unauthenticated backoffice | Middleware + per-action re-authentication |

| Unsafe HTML | No `dangerouslySetInnerHTML` without sanitization; all user content rendered via React (auto-escaped) |

Input validation (Zod)

All Server Action inputs are validated with Zod before any database operation. Never trust the shape of data coming from the client.

```
const CreateCustomerSchema = z.object({
  full_name: z.string().min(1).max(100).trim(),
  phone:     z.string().max(20).optional().nullable(),
  email:     z.string().email().optional().nullable().or(z.literal('')),
  address:   z.string().max(255).optional().nullable(),
  notes:     z.string().max(1000).optional().nullable(),
})

const CreateOrderSchema = z.object({
  customer_id: z.string().uuid(),
  source:      z.enum(['WALK_IN', 'PORTAL']),
  assigned_to: z.string().uuid().optional().nullable(),
  due_at:      z.string().datetime().optional().nullable(),
  notes:       z.string().max(1000).optional().nullable(),
  items: z.array(z.object({
    service_id: z.string().uuid(),
    quantity:   z.number().positive().multipleOf(0.001),
  })).min(1),
})

const AdvanceStatusSchema = z.object({
  order_id: z.string().uuid(),
  to_status: z.enum(['WASHING', 'DRYING', 'FOLDING', 'READY']),
  note:     z.string().max(500).optional(),
})

const CancelOrderSchema = z.object({
  order_id: z.string().uuid(),
  cancel_reason: z.string().min(1).max(500),
})
```

17. Validation Rules

Customers

- `full_name`: required, 1–100 characters
- `phone`: optional, max 20 characters, no format enforcement (international-friendly)
- `email`: optional, valid email format if provided
- `address`: optional, max 255 characters
- `notes`: optional, max 1000 characters
- Duplicate phone: show warning dialog, allow override

Services

- `name`: required, 1–100 characters, unique (application-level warning)
- `pricing_unit`: required, one of `PER_KG` | `PER_PIECE`
- `unit_price_cents`: required, integer ≥ 0 , max 99,999,999 ($\approx$ $\blacksquare$ 999,999.99)
- `sort_order`: integer, default 0

Orders

- customer_id: required, valid UUID, must exist in customers
- source: required, WALK_IN | PORTAL
- due_at: optional, must be in the future
- notes: optional, max 1000 characters
- At least 1 item required
- total_cents is computed server-side (not trusted from client)

Order items

- service_id: required, must be an active service
- quantity: positive number, max 3 decimal places
- line_total_cents: computed server-side = $\text{ROUND}(\text{unit_price_cents} \times \text{quantity})$

Status transitions

- Forward: exactly one step forward in the workflow
- Backward: exactly one step backward; note required
- Cancellation: cancel_reason required, min 1 char
- Completion: only from READY status

Money

- All monetary values stored as integer centavos
- No floating-point arithmetic in money calculations
- UI displays as: ₱1,234.56 (Philippine peso, comma thousands separator, 2 decimal places)

18. Caching and Data-Fetching Strategy

Caching by resource type

Resource	Cache strategy
Marketing page	export const revalidate = 3600 (ISR, 1 hour)
Active services list	unstable_cache(['services'], { revalidate: 300 }) — invalidated on service mutation
Dashboard aggregates	unstable_cache(['dashboard'], { revalidate: 60 })
Reports data	unstable_cache(['reports'], { revalidate: 60 })
Orders list	No cache (operational, real-time data)
Order detail	No cache
Customer data	No cache (PII)
Portal orders	No cache (PII, real-time)
Public tracking	No shared cache (per-code response, no shared state risk)

Cache invalidation

After every mutation, call `revalidatePath()` or `revalidateTag()`:

```
// After order status change:
revalidatePath(`/orders/${orderId}`)
revalidatePath('/orders')
revalidatePath('/dashboard')

// After service edit:
revalidateTag('services')
revalidatePath('/services')
```

Pagination

- Default page size: 20 rows
- All list pages use URL query param `?page=N`
- No cursor pagination needed at MVP scale (keyset pagination if performance issues arise)

19. Performance Strategy

Core principles

Solve actual bottlenecks. Do not add optimization theater.

Strategies in use

- **Server Components by default** — no hydration cost for read-heavy pages
- **Selective column queries** — never `SELECT *` in production code
- **Database indexes** — all FK columns and commonly filtered columns indexed (see Section 9)
- **N+1 prevention** — use Supabase nested selects to load order items and events in one query
- **Pagination** — all list views paginated (20/page)
- **Image optimization** — Next.js `<Image>` for any marketing images
- **Lazy loading** — Aceternity and Magic UI effects are lazy-loaded / conditionally rendered
- **Static marketing** — Marketing page statically generated or ISR

What is NOT done

- No React Query / SWR
- No global client state
- No aggressive prefetching
- No WebSocket / real-time subscriptions
- No service worker / offline caching

20. Responsive and Accessibility Requirements

Responsive breakpoints

- Mobile-first design
- shadcn/ui's built-in responsive utilities
- Tailwind breakpoints: sm (640px), md (768px), lg (1024px), xl (1280px)

Desktop backoffice

- Sidebar navigation (collapsible)
- Data tables with sortable columns
- Horizontal workflow stepper for order status
- Dense but readable information density

Mobile

- Bottom navigation or hamburger menu
- Vertical workflow stepper
- Cards instead of tables for order/customer lists
- Touch-friendly action buttons (minimum 44px tap target)

- No horizontal scrolling on main content

Accessibility

- Semantic HTML throughout (<main>, <nav>, <header>, <section>, <button>, etc.)
- ARIA labels on icon-only buttons
- Visible focus rings (never outline: none without a visible replacement)
- Dialogs and sheets use role="dialog", aria-modal, and focus trap
- Color is never the sole means of conveying status (always pair with text or icon)
- prefers-reduced-motion: all animations wrapped in @media (prefers-reduced-motion: no-preference) or checked via hook
- Contrast ratios: minimum WCAG AA (4.5:1 for text)
- Form inputs have associated <label> elements

21. UI Library Usage Rules

shadcn/ui

Primary UI component library. Use for all standard UI elements.

Use shadcn for: buttons, inputs, dialogs, sheets, dropdowns, selects, tables, badges, cards, alerts, toasts, date pickers, navigation, sidebars, tabs, accordions.

Do NOT create custom versions of: anything shadcn already provides.

Extend via: className overrides and composition. Not wrappers around every primitive.

Magic UI

Use for tasteful micro-interactions in the application UI.

Appropriate uses:

- Animated counters on dashboard stat cards
- Subtle shimmer or border effects on focused elements
- Success animations (e.g., order completion)
- Number reveal animations on totals

Do NOT use for: heavy motion in operational forms, anything that delays task completion, decorative motion without purpose.

Aceternity UI

Use selectively, marketing pages only.

Appropriate uses:

- Hero section scroll storytelling
- Animated product showcase (order flow visualization)
- Selected bento or feature grid sections
- Background texture or subtle gradient effects in hero

Never use in: backoffice pages, portal pages, data tables, forms, dialogs.

Avoid: excessive particles, aurora backgrounds, giant glowing orbs, full-screen glassmorphism, 3D card flips, infinite marquees.

Custom components

Only build custom components for product-specific behavior:

| Component | Purpose |

|---|---|

| OrderFlowStepper | Visual representation of order workflow stages |

| `StatusBadge` | Colored + labeled badge for `order_status` values |

| `EmptyState` | Consistent empty state with illustration and CTA |

| `PageHeader` | Consistent page title + breadcrumb + action area |

22. Marketing Website Specification

Route: /

Narrative structure

1. **Hero** — "Every load has a flow." — Full-screen hero, product wordmark, animated subtitle. CTA: "See it in action" (scrolls down) + "Sign In" (→ /login).
2. **Order flow story** — Animated visualization of the 6-stage workflow (RECEIVED → COMPLETED). Each stage appears as the user scrolls.
3. **Dashboard preview** — Screenshot or interactive mockup of the backoffice dashboard. Subtle shimmer/reveal animation.
4. **Features** — 3-column bento grid: Real-time tracking, Team workflow, Customer portal, Audit trail, Order history, Service catalog.
5. **How it works** — 3-step visual: Staff receives → System tracks → Customer picks up.
6. **Value proposition** — Focused copy on operational clarity, no complexity.
7. **CTA** — Final call-to-action to sign in or access tracking.

Technical notes

- Marketing page is a Server Component with Client islands for animations
- Aceternity effects are lazy-loaded and respect `prefers-reduced-motion`
- No authentication required
- No database queries on the marketing page
- Public tracking link prominently accessible: "Track your laundry →"

23. Loading, Empty, Error, and Success States

Every meaningful UI area must handle these states:

Loading

- Skeleton UI for page-level data (Server Component suspense boundaries)
- Spinner inside action buttons (disable button immediately on click, show spinner)
- Prevent duplicate submissions: button disabled while action is in-flight

Empty

- Customer list: "No customers yet. Add your first customer." + CTA button
- Orders list: "No orders match your filters." or "No orders yet."
- Order items: "Add at least one service." (blocked from saving without items)
- Status history: Single entry always exists (initial RECEIVED event)

Error

- Form validation errors: Inline field-level errors
- Server Action failures: Toast notification with error message
- Not found (404): Consistent `/not-found.tsx` for each route group
- Unauthorized: Redirect to appropriate page, not raw error

Success

- Create customer: Toast "Customer created." + navigate to customer detail
- Create order: Toast "Order created." + navigate to order detail
- Status advance: Toast "Order moved to [status]." + page revalidates
- Order completion: Toast "Order completed and marked as paid." + completion state shown
- Destructive action confirmation: Modal/dialog with "Are you sure?" before proceeding

24. Testing Strategy

Approach (academic MVP scope)

Full automated test coverage is out of scope for the MVP. The strategy is:

1. **Manual test checklist** — Validate each item in the Definition of Done (Section 28) manually before feature freeze.
2. **Seed data** — A comprehensive seed script (supabase/seed . ts) that creates demo data covering all states and scenarios.
3. **Defense demo path** — The end-to-end demo flow (Section 25) is manually tested and verified to work completely before defense.

Seed data requirements

The seed script must create:

- 1 admin user
- 2 staff users
- 1 customer user (with linked customer record)
- 5–8 customer records (mix of walk-in and linked)
- 4–6 services (mix of PER_KG and PER_PIECE)
- 10–15 orders in various statuses (at least one in each status including COMPLETED and CANCELLED)
- Status history for all orders
- A demo order that is in READY status (ready to demonstrate completion)

Post-freeze (if time allows)

- Playwright E2E test for the defense demo path
- Basic Vitest unit tests for order - machine . ts (state transition logic)

25. Defense and Demo Flow

The system must support this exact walkthrough without errors:

1. **Open /** — marketing page loads, animations render correctly
2. **Scroll through product story** — order flow visualization, feature highlights visible
3. **Click "Sign In"** → /login — Clerk login form appears
4. **Log in as admin** — redirected to /dashboard
5. **Dashboard shows** — order counts, recent orders, at-a-glance stats
6. **Navigate to /customers** — customers list with search
7. **Create new customer** — enter name and phone; trigger duplicate warning if applicable
8. **View created customer** — customer detail page with order history (empty)
9. **Navigate to /orders/new** — order creation form
10. **Select the customer** — customer search/select works
11. **Add 2 services** — select service, enter quantity; totals calculated live
12. **Submit** — order created, redirected to order detail
13. **Order detail shows** — order number, tracking code, RECEIVED status, items, total
14. **Advance to WASHING** — status updates, stepper reflects new state, event logged

15. **Advance through DRYING, FOLDING, READY** — each transition works
16. **Show status history** — all 5 events visible with timestamps and actor
17. **View order slip** at `/orders/[orderId]/slip` — printable layout
18. **Complete the order** — "Complete & Mark Paid" button → order is COMPLETED, `paid_at` set
19. **Verify completed state** — `paid_at` timestamp visible, no further actions available
20. **Log out** — Clerk sign-out
21. **Log in as customer** — redirected to `/portal/orders`
22. **Portal shows own orders** — customer sees the completed order
23. **Open order detail** — portal view shows status history (customer-friendly labels)
24. **Log out** — sign out from portal
25. **Visit** `/track` — enter tracking code from the demo order
26. `/track/[code]` **shows** — order status, no PII, clean public view
27. **Attempt to access** `/dashboard` **unauthenticated** — redirect to login
28. **Attempt to access another customer's portal order** — 404 or same-as-not-found response

26. Explicit MVP Exclusions

The following are permanently out of scope for the MVP. Do not implement, plan for, or partially scaffold:

- Payment gateways, payment providers (Stripe, GCash, Maya, PayPal)
- Payment transaction table or payment methods
- SMS notifications
- Email notification infrastructure (transactional emails, order confirmations)
- Delivery / logistics tracking
- Inventory management
- Loyalty programs or promotions
- Multi-branch / multi-location support
- AI chatbot or AI-assisted features
- Real-time WebSocket subscriptions or live-updating UI
- Drag-and-drop order board
- PWA / offline support
- Garment photo management
- Per-garment tracking
- Barcode or QR scanning
- External webhooks
- Mobile native app
- `Payment_note` field on orders
- Partial payment tracking
- Refund tracking

27. Finite Implementation Roadmap

Phase 0 — Foundation (before any feature code)

- [] Initialize Next.js project (TypeScript, Tailwind, App Router)
- [] Install and configure Clerk
- [] Install and configure Supabase (@supabase/ssr)
- [] Install shadcn/ui

- [] Create database schema (migrations)
- [] Set up route groups and layout structure
- [] Establish DAL pattern (reference `getOrderById`)
- [] Establish Server Action pattern (reference `advanceOrderStatus`)
- [] Middleware configuration
- [] `lib/money.ts`, `lib/time.ts`, `lib/order-machine.ts`
- [] Seed script skeleton

Phase 1 — Core backoffice (staff-facing operations)

- [] Auth: Clerk login/logout flow
- [] Team onboarding: admin can invite staff, set role
- [] Services: CRUD (admin only)
- [] Customers: Create, List, View, Edit, Archive
- [] Orders: Create with items (RECEIVED)
- [] Orders: List with filters
- [] Orders: Detail view
- [] Orders: Status advancement (forward)
- [] Orders: Backward transition with note
- [] Orders: Cancellation
- [] Orders: Completion (atomic `paid_at` + COMPLETED)
- [] Order slip: printable layout
- [] Dashboard: basic stats

Phase 2 — Customer portal and tracking

- [] Customer portal layout and auth guard
- [] Portal: orders list (own only)
- [] Portal: order detail
- [] Portal: profile view/edit
- [] Customer account linking (email match at first login)
- [] Public tracking: `/track` search form
- [] Public tracking: `/track/[code]` status view

Phase 3 — Reports and settings

- [] Reports: daily/weekly order summary
- [] Reports: revenue summary (completed orders)
- [] Reports: orders by service
- [] Settings: basic system configuration (shop name, etc.)

Phase 4 — Polish and defense readiness

- [] Marketing homepage (full narrative)
- [] Loading states (skeletons) for all pages
- [] Empty states for all lists
- [] Error states and error boundaries
- [] Mobile responsive review
- [] Accessibility review (focus, ARIA, keyboard)
- [] Seed data completion
- [] End-to-end defense demo walkthrough
- [] Bug fixes

28. Definition of Done / Feature-Freeze Criteria

Feature-level DoD

A feature is complete only when:

- ☐ UI renders correctly
- ☐ Mobile responsive behavior works
- ☐ Loading state exists
- ☐ Empty state exists
- ☐ Error state exists
- ☐ Success feedback (toast or navigation)
- ☐ Form validation works (client-side: UX; server-side: security)
- ☐ Authorization is enforced in the Server Action / DAL
- ☐ Database constraints are correct
- ☐ Cache/path invalidation works after mutation
- ☐ Accessibility is acceptable (keyboard nav, ARIA, focus)
- ☐ Edge cases handled (duplicate detection, empty orders, etc.)
- ☐ No console errors or unhandled exceptions

Feature freeze criteria (all must pass)

- ☐ Authentication works (login, logout, session persistence)
- ☐ Authorization works (role enforcement, IDOR prevention tested)
- ☐ Customers CRUD complete
- ☐ Services CRUD complete
- ☐ Orders CRUD complete (create, view, list, items management)
- ☐ Order state machine works end-to-end
- ☐ Backward transitions with note work
- ☐ Cancellation with reason works
- ☐ Completion (paid_at atomic) works
- ☐ Status history shown correctly
- ☐ Order slip renders and prints
- ☐ Customer portal works (own orders, own profile)
- ☐ Public tracking works
- ☐ Dashboard shows real data
- ☐ Reports show real data
- ☐ Responsive UI tested on mobile viewport
- ☐ Seed data script works and produces a complete demo dataset
- ☐ Full defense demo walkthrough completes without errors

29. Remaining Decisions

The following decisions were resolved autonomously and are now locked:

| Decision | Resolution |

|---|---|

| Order number format | LF - NNNNN via Postgres sequence |

| Tracking code format | 16-char nanoid (URL-safe) |

| quantity column type | NUMERIC(10,3) |
| order . source values | WALK_IN \| PORTAL |
| Customer portal link flow | Email-match auto-link at first portal login |
| Role storage | Clerk publicMetadata.role + Supabase users.role (dual write) |
| Reports scope | Order summary by date range, revenue by period, orders by service |
| Page size | 20 rows |
| Money format | ■1,234.56 |
| Timezone | Asia/Manila (all display), UTC (all storage) |
| Payment behavior | Staff completes order → atomic paid_at + COMPLETED (no payment subsystem) |

One genuine remaining product decision:

> **Settings page scope:** What should /settings allow the admin to configure? Candidate items: shop name (displayed on slips), default due date window (e.g., 3 days after received), pagination size preference. Define before Phase 3.

No other implementation decisions require your input before production coding begins.